MINI ASSESSMENT

A Friendly Rule-Based Python Chatbot

STUDENT	NARU MAHESH KUMAR REDDY
REGISTER NUMBER	99230040146
DEPARTMENT	CSE (AI & ML)
INSTITUTION	Kalasalingam Academy of Research and Education
DATE	11 September 2026

PROJECT DESCRIPTION

A structured Python-based mini project demonstrating rule-based conversational interaction through input processing, conditional decision-making, and controlled response generation.

INPUT → PROCESS → RESPOND → REPEAT → EXIT

01 | PROJECT OVERVIEW

1. AIM

To develop a simple interactive rule-based chatbot in Python that continuously accepts user messages, identifies greetings and common questions using conditional statements, provides suitable responses, and terminates the conversation when the user enters an exit command.

2. CONCEPT

A rule-based chatbot works through predefined rules. The program checks the user's input, identifies a matching condition, and produces the response associated with that condition. It does not learn from the conversation; its behaviour is determined by the rules defined by the programmer.

Basic flow: User Input → Rule Matching → Predefined Response

3. WORKING

The chatbot first asks for the user's name and then continuously accepts messages. Each message is normalized using **lower()** and **strip()**. The program checks for greetings, Python, AI, study tips, help, and exit commands using conditional statements. If no condition matches, a suitable fallback response is displayed.

This approach keeps the system simple, predictable, and easy to test while demonstrating the basic logic used in conversational programming.

4. KEY REQUIREMENTS

- Continuous user input using a while loop.
- Input cleaning using lower() and strip().
- Recognition of multiple greetings and conversation topics.
- Personalized responses using the user's name.
- Fallback handling for unknown input.
- Controlled termination using bye or exit and break.

02 | SYSTEM DESIGN

Functional Architecture

The chatbot follows a sequential decision process. User input is collected, normalized, checked against predefined rules, and converted into a response. The loop then waits for the next message until an exit command is received.

USER	→	INPUT	→	PROCESS	→	RULE CHECK	→	RESPONSE
		Message		Normalize		Condition		Output

Conversation Rules

Category	Example	System behaviour
Greeting	hello / hi / hey	Return a personalized welcome.
Python	tell me about Python	Give a short introduction to Python.
AI	what is AI	Explain AI in simple terms.
Study	give me a study tip	Provide practical programming guidance.
Help	help / what can you do	List supported topics.
Identity	are you a real AI	Explain the rule-based nature of the system.
Exit	bye / exit	Print goodbye and terminate.
Unknown	xyz / unrelated text	Display a useful fallback response.

Engineering Characteristics

- **Predictability:** the same input follows the same predefined rule.
- **Traceability:** each response can be traced to a specific condition.
- **Maintainability:** new topics can be added through additional rules.
- **Usability:** personalized responses make the interaction easier to follow.

03 | IMPLEMENTATION

Python Implementation

The implementation uses introductory Python features so that the complete decision logic remains easy to understand and reproduce.

```
def chatbot():
    print("Your friendly coding companion")
    name = input("What is your name? ").strip()

    print(f"Nice to meet you, {name}!")
    print("Ask me about Python, AI, study tips, or type bye to stop.")

    while True:
        user = input("You: ").lower().strip()

        if user in ["bye", "exit"]:
            print(f"Bot: Goodbye, {name}! Keep coding and keep learning.")
            break

        elif "hello" in user or "hi" in user or "hey" in user:
            print(f"Bot: Hello, {name}! What would you like to explore?")

        elif "python" in user:
            print("Bot: Python is easy to learn and useful in AI,")
            print("automation, data analysis, and web development.")

        elif "real ai" in user:
            print("Bot: I am rule-based. My replies come from")
            print("predefined rules rather than machine learning.")

        elif "ai" in user or "artificial intelligence" in user:
            print("Bot: AI helps computers perform tasks that usually")
            print("need human-like learning, reasoning, or decisions.")

        elif "study" in user or "tip" in user:
            print("Bot: Try one small Python problem every day.")
            print("Regular practice is better than cramming.")

        elif "help" in user:
            print("Bot: Ask about Python, AI, study tips, greetings,")
            print("or type bye to end the chat.")

        else:
            print("Bot: I do not have a rule for that yet.")
            print("Try Python, AI, study tips, help, or a greeting.")

chatbot()
```

Implementation Logic

- **Initialization:** introduces the chatbot and collects the user's name.
- **Input handling:** normalizes every message with lower() and strip().
- **Decision layer:** uses if/elif/else to compare messages with predefined rules.
- **Loop control:** while True enables repeated interaction and break ends it.
- **Fallback:** the final else handles unsupported messages.

04 | VALIDATION & DEMONSTRATION

Test Strategy

Representative conversational cases are used to verify greetings, topic recognition, input normalization, unknown input, and both exit commands.

ID	Input	Expected behaviour	Status
TC-01	hello	Personalized greeting	PASS
TC-02	HELLO	Greeting recognized after normalization	PASS
TC-03	tell me about Python	Python information displayed	PASS
TC-04	what is AI	AI explanation displayed	PASS
TC-05	give me a study tip	Study guidance displayed	PASS
TC-06	help	Supported topics displayed	PASS
TC-07	are you a real AI	Identity response displayed	PASS
TC-08	xyz	Fallback response displayed	PASS
TC-09	bye	Goodbye displayed and loop ends	PASS
TC-10	exit	Goodbye displayed and loop ends	PASS

Sample Execution

```
Your friendly coding companion
What is your name? NARU MAHESH KUMAR REDDY
Nice to meet you, NARU MAHESH KUMAR REDDY!

You: hello
Bot: Hello, NARU MAHESH KUMAR REDDY! What would you like to explore?

You: tell me about python
Bot: Python is easy to learn and useful in AI, automation,
data analysis, and web development.

You: what is ai
Bot: AI helps computers perform tasks that usually need
human-like learning, reasoning, or decisions.

You: are you a real ai
Bot: I am rule-based. My replies come from predefined
rules rather than machine learning.

You: give me a study tip
Bot: Try one small Python problem every day.
Regular practice is better than cramming.

You: xyz
Bot: I do not have a rule for that yet. Try Python, AI,
study tips, help, or a greeting.

You: bye
Bot: Goodbye, NARU MAHESH KUMAR REDDY! Keep coding and keep learning.
```

05 | CONCLUSION

Project Outcome

The project demonstrates how a basic Python program can be organized into a clear conversational workflow. It covers continuous input, input cleaning, multiple greetings, multiple topics, conditional decision making, personalized output, an unknown-input response, and a controlled exit.

Its main engineering value is transparency: every conversational behaviour is represented by a rule, so the logic can be understood, tested, and extended without requiring a complex framework.

Key Takeaways

- Simple programming constructs can create a structured conversational system.
- Input normalization improves consistency when users type the same idea differently.
- Predefined rules make chatbot behaviour predictable and easy to test.
- A fallback branch handles messages outside the known rule set.
- The design can be extended by adding new topics and corresponding conditions.

Final System Flow

INPUT → PROCESS → RESPOND → REPEAT → EXIT

NARU MAHESH KUMAR REDDY · 99230040146